



Advanced Trees

CEN207 Data Structures — Week 11

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	BST: insert, search, delete, why balance matters
2	AVL (4 rotations, insert, delete) · red-black · splay
3	2-3 tree · segment tree · Fenwick tree · choosing a technique

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
BST insert/search/delete, degenerate case	Section 1
AVL, red-black, splay, 2-3 tree	Sections 2–5
Segment tree, Fenwick tree	Sections 6–7
Choosing a technique	Section 8

How the code examples work

- Every idea has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-11/c/` and `code/week-11/java/`
- Each program's expected output is in the week notes

Bridge from Week 4

- Week 4 gave you **tree vocabulary**: root, leaf, depth, height, subtree
- Week 4 gave you **traversals**: inorder, preorder, postorder, level-order
- Week 4's heap was never **ordered** left-vs-right, only parent-vs-child
- Today's trees add an ordering rule for the first time

Recap — Week 4: height and depth

- **Depth** of a node = edges from the root to it (root = 0)
- **Height** of a tree = greatest depth of any node
- Empty tree: height -1 by convention; one node: height 0
- Today: height is the single number that decides every cost

Map of the week — the one rule

- A **binary search tree**: left subtree smaller, right subtree larger
- That rule alone gives fast search — IF the tree stays shallow
- Four different strategies keep it shallow: AVL, red-black, splay, 2-3
- Two more trees answer **range** questions instead of single-key ones

Map of the week — at a glance

Balanced BSTs	Range-query trees
AVL — strict balance factor	Segment tree — build once, query $O(\log n)$
Red-black — looser, color-based	Fenwick tree — one array, i & $-i$
Splay — no balance, adapts to use	—
2-3 tree — grows only at the root	—

Recap — Week 4: traversals

- **Inorder**: left, visit, right — sorted order on a BST
- **Preorder**: visit, left, right — copies a tree's shape
- **Postorder**: left, right, visit — safe for deleting a tree
- **Level-order**: an explicit queue, breadth-first

Vocabulary check

Term	Meaning
Balance factor	$\text{height}(\text{left}) - \text{height}(\text{right})$
Rotation	$O(1)$ local pointer rearrangement
Amortized cost	averaged over a long operation sequence
Invertible operation	undoable via subtraction (sum, not min)

1. The Binary Search Tree

A question to start

Week 1: sorted array, binary search — fast, but inserting costs $O(n)$.

Week 2: linked list — insert $O(1)$, but search needs $O(n)$.

Is there a structure that does BOTH better than $O(n)$?

A short history

- BST ideas appear independently **1959–1962**
- P. F. Windley, A. D. Booth & A. J. T. Colin, T. N. Hibbard
- **Hibbard, 1962** — usually credited for working out deletion
- Deletion is exactly what section 1.4 covers today

Intuition — a phone book, but a tree

- Open to the middle page: is your name before or after it?
- Keep halving — that is binary search on an array
- A BST bakes that same halving into the **shape** of the structure
- Rule at every node: left smaller, right larger

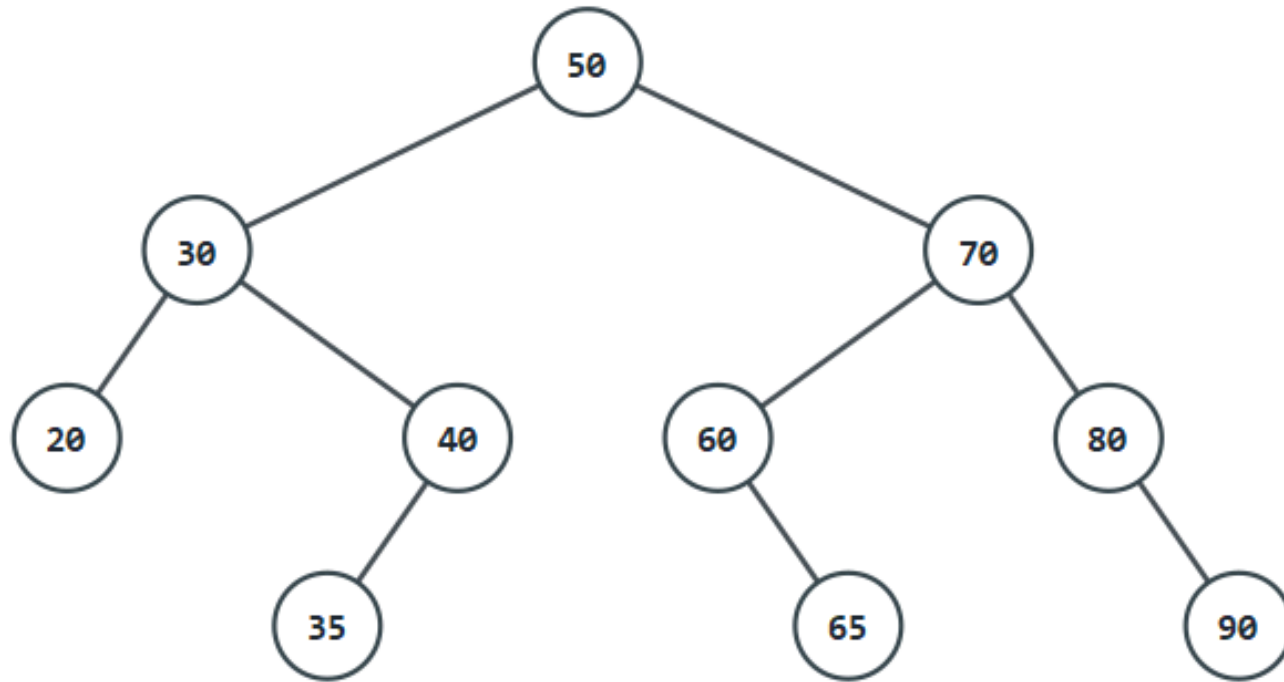
The BST ADT

Operation	What it does	Complexity
<code>insert(key)</code>	Adds key; duplicate ignored	$O(h)$
<code>search(key)</code>	Reports present or not	$O(h)$
<code>delete(key)</code>	Removes key if present	$O(h)$
<code>min / max</code>	Smallest / largest key	$O(h)$

In memory — insert's idea

- Walk down from the root, comparing `key` at every node
- Smaller → go left; larger → go right; equal → duplicate, stop
- Reach a `NULL` child → that is the new node's spot
- Link it in; nothing else in the tree moves

Binary search tree: insert



to insert:

50	30	70	20	40
60	80	35	65	90

Done: 10 keys processed. Every insertion makes as many comparisons as the tree's height — $O(\log n)$ in a balanced tree, but it can rise to $O(n)$ for an unlucky order (see: bst-degenerate).

Code — bst_insert()

```
Node *bst_insert(Node *root, int key) {  
    Node *cur = root, *parent = NULL;  
    while (cur != NULL) {  
        parent = cur;  
        if (key == cur->key) return root;  
        if (key < cur->key) cur = cur->left;  
        else cur = cur->right;  
    }  
    Node *n = malloc(sizeof(Node));  
    n->key = key; n->left = NULL; n->right = NULL;
```

Code — bst_insert(), linking in

```
if (parent == NULL) return n;  
if (key < parent->key) parent->left = n;  
else                    parent->right = n;  
return root;  
}
```

Expected output

```
insert(50): inorder = 50  height = 0  
insert(30): inorder = 30 50  height = 1  
insert(70): inorder = 30 50 70  height = 1  
insert(20): inorder = 20 30 50 70  height = 2
```

Why insert is $O(h)$

- One comparison per level visited, at most
- Never revisits a node once passed
- h small (balanced) $\rightarrow$ fast; h large (chain) $\rightarrow$ slow
- Section 1.4 shows exactly how large h can get

Common mistake

- Forgetting to `root = bst_insert(root, key);`
- Without reassigning, the caller's `root` never updates
- C/Java pass pointers/references **by value** here
- The function must **return** the (possibly new) root

Mini question

In the animation's "ascending order" edge case, every new key becomes a RIGHT child. Why never a left one?

Mini answer

- Every later key is larger than everything already inserted
- So every comparison during the walk says "go right"
- The tree leans entirely to the right — a chain
- This is exactly section 1.4's topic, next

A question to start — search

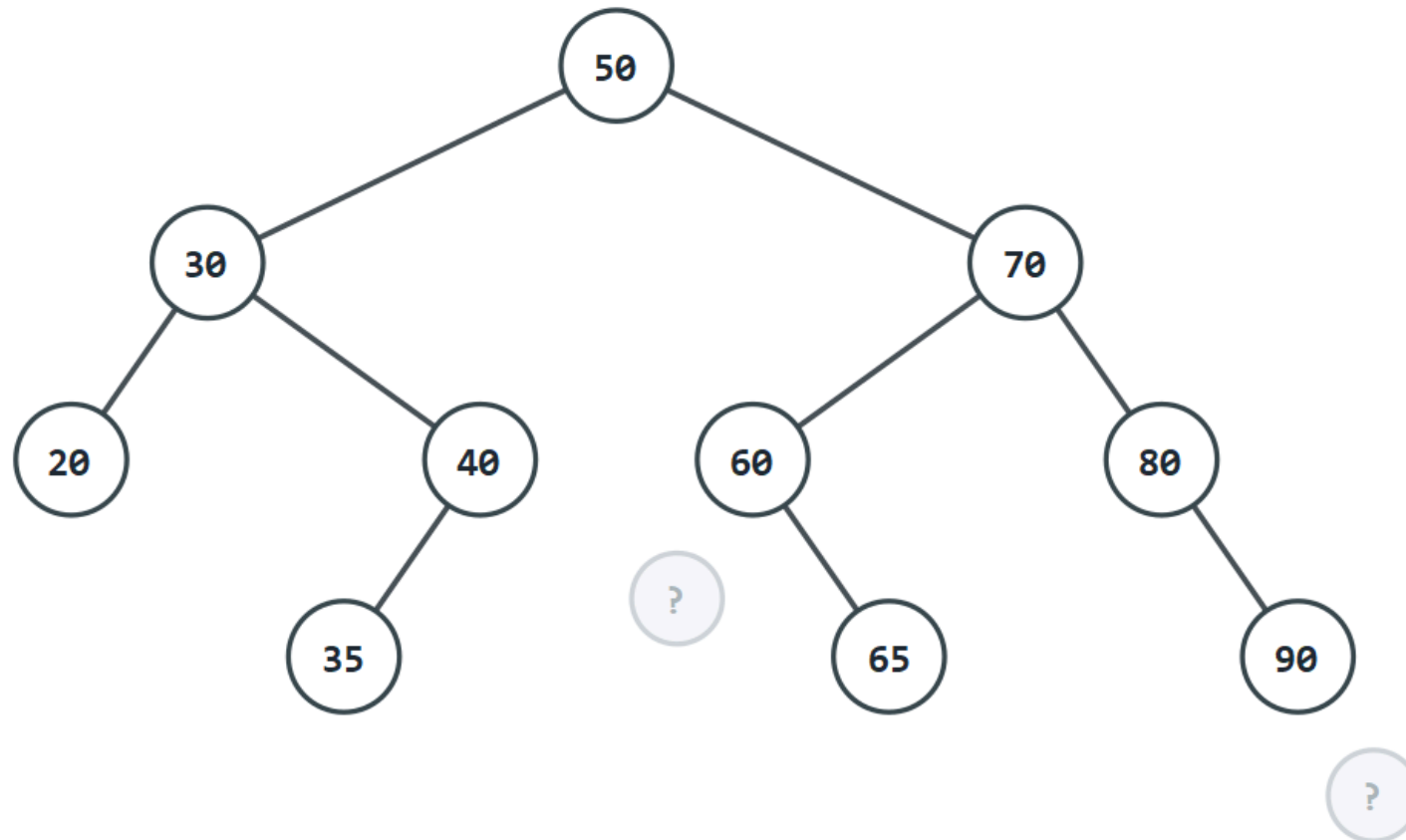
`insert` already walks down comparing keys.

`search` is almost the same walk — how many nodes, worst case?

The idea — search

- Equal → found; smaller → only the left subtree can have it
- The ordering rule GUARANTEES the right subtree cannot
- Larger → mirror image
- NULL reached before a match → not present

Binary search tree: search



Done: 5 searches completed. Every search makes at most as many comparisons as the tree's height.

Code — bst_search()

```
int bst_search(Node *root, int key) {
    Node *cur = root;
    probes = 0;
    while (cur != NULL) {
        probes++;
        if (key == cur->key) return 1;
        if (key < cur->key) cur = cur->left;
        else                cur = cur->right;
    }
    return 0;
}
```

Expected output

```
search(65): found, 4 probes  
search(20): found, 3 probes  
search(55): not found, 3 probes  
search(100): not found, 4 probes
```

Why search is $O(h)$

- Same one-comparison-per-level argument as insert
- Best case $O(1)$: the root itself
- Worst case: as deep as the tree gets
- Again: everything hinges on h

Common mistake

- Treating "not found" as an error condition
- It is a normal, expected outcome, not a crash
- The `while (cur != NULL)` guard handles it cleanly
- Continuing to compare after a match (off-by-one) also wastes work

Mini question

search(10) on a 10-node ascending-insert chain costs 10 probes.
search(1) costs only 1. Why such a big gap?

Mini answer

- That tree is a pure chain: 1 at the root, 10 at the deepest leaf
- Root's own key: one comparison
- Deepest leaf: one comparison per level down to it
- height + 1 comparisons, worst case

A question to start — delete

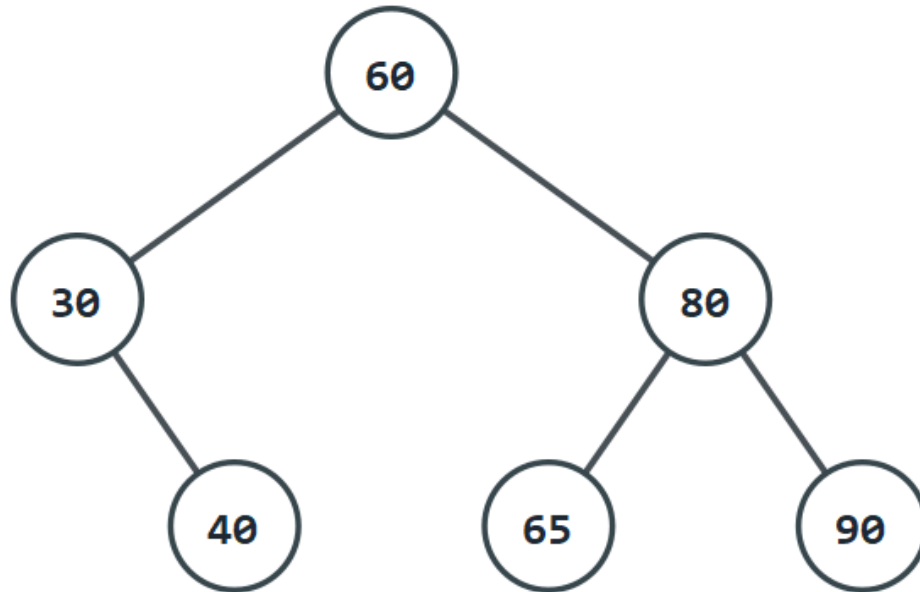
Deleting a **leaf** is easy: detach it.

Deleting a node with **two children** is not. Why not?

The idea — three cases

Case	Fix
Leaf	Detach it
One child	Parent links directly to the child
Two children	Copy in the in-order successor's key, then delete IT

Binary search tree: delete



Done: 4 deletes processed. The remaining tree is still a valid BST.

Code — bst_delete(), finding the node

```
Node *bst_delete(Node *root, int key) {  
    Node *cur = root, *parent = NULL;  
    while (cur != NULL && key != cur->key) {  
        parent = cur;  
        cur = (key < cur->key) ? cur->left : cur->right;  
    }  
    if (cur == NULL) return root;
```

Code — bst_delete(), two children

```
if (cur->left != NULL && cur->right != NULL) {  
    Node *succ = cur->right, *succParent = cur;  
    while (succ->left != NULL) {  
        succParent = succ; succ = succ->left;  
    }  
    cur->key = succ->key;  
    parent = succParent; cur = succ;  
}
```

Expected output

```
delete(35): inorder = 20 30 40 50 60 65 70 80 90  
delete(70): inorder = 20 30 40 50 60 65 80 90  
delete(50): inorder = 20 30 40 60 65 80 90  
delete(20): inorder = 30 40 60 65 80 90
```

Why delete is $O(h)$

- Finding the node: $O(h)$
- Finding a successor: at most another $O(h)$
- Never revisits nodes already on the search path
- Same worst case as insert and search

Common mistake

- Copying the successor's key but forgetting to delete ITS node
- The key now exists twice in the tree
- Successor vs. predecessor: either works, but stay consistent
- Forgetting `free()` in C — a real memory leak

Mini question

Why is a BST delete's successor guaranteed to have
AT MOST one child?

Mini answer

- Successor = leftmost node of the right subtree
- Leftmost means: no left child, by definition
- May or may not have a right child
- Never both — so it is always case 1 or case 2

A question to start — degenerate

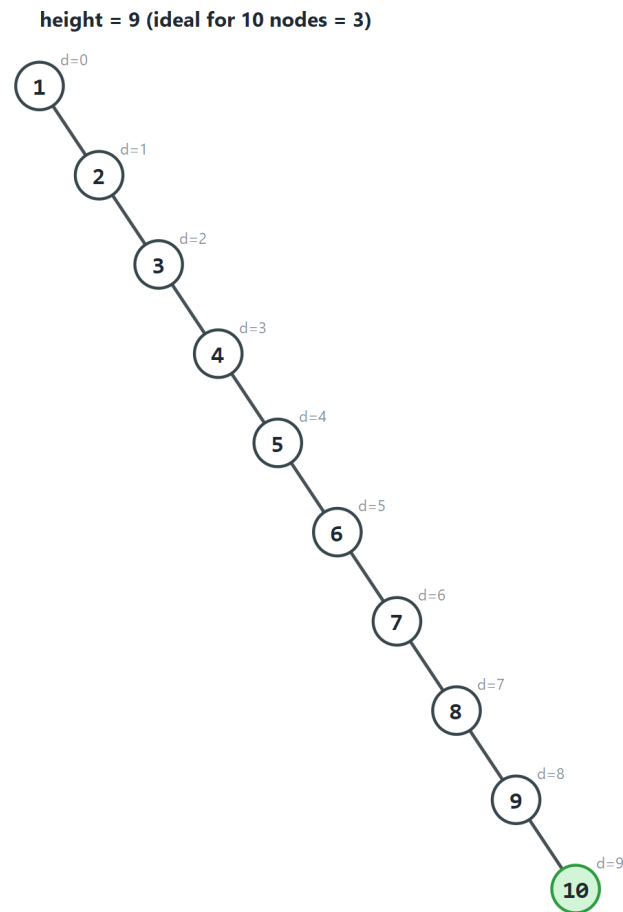
Every section so far states cost as $O(h)$.

What IS h , concretely, for n keys in an arbitrary order?

The idea — order, not just the key set

- Every technique compares VALUES, never looks at tree SHAPE
- Sorted input: every new key is bigger than everything so far
- Every new key attaches one level deeper than the last
- The tree **degenerates** into a chain: height $n - 1$

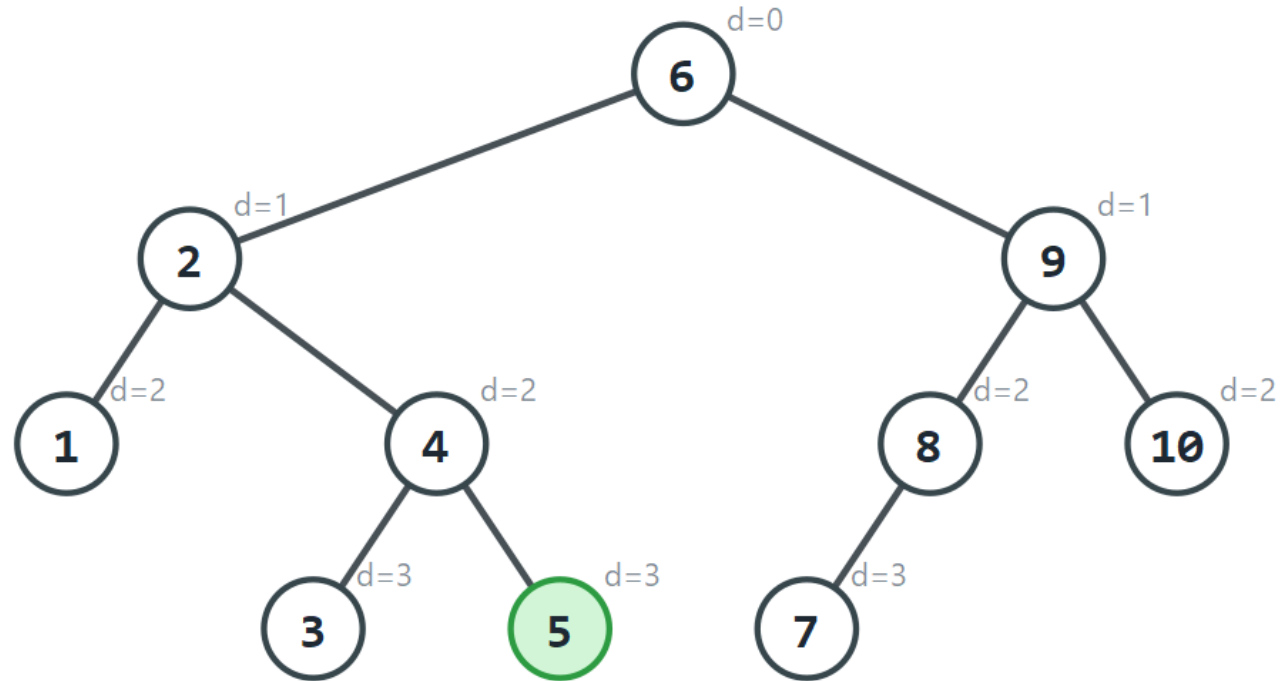
Why balancing matters



Done. 10 nodes, height = 9; a balanced tree would need only height 3 (about 3.0x apart). This is exactly why BALANCED trees — AVL, red-black — exist, coming up next.

The same keys, shuffled

height = 3 (ideal for 10 nodes = 3)



Done. 10 nodes, height = 3; a balanced tree would need only height 3 (about 1.0x apart). This is exactly why BALANCED trees — AVL, red-black — exist, coming up next.

Expected output

```
insert(10): height = 9  (ideal for 10 nodes = 3)
final: n = 10, height = 9, ideal = 3
```

vs. the shuffled edge case: `final: n = 10, height = 3, ideal = 3`

Why this matters for complexity

- Worst case height: $n - 1$ (sorted or reverse-sorted input)
- Every operation from sections 1.1–1.4 becomes $O(n)$
- Average case (random order): $O(\log n)$ — but "random" is not guaranteed
- Sorted input is common in practice: imports, replayed logs

Common mistake

- Assuming "it's a BST" means $O(\log n)$ automatically
- Benchmarking only with random test data
- Random data HIDES this exact failure mode
- Only a SELF-BALANCING BST guarantees $O(\log n)$ worst case

Mini question

You must build a BST from data you KNOW is already sorted.
Cheapest fix, without switching tree types?

Mini answer

- Shuffle the keys into random order before inserting
- Or: recursively pick the middle element as root, recurse both halves
- Builds a perfectly balanced tree directly, $O(n)$, no rotations
- Sections 2–5 solve the GENERAL problem automatically

2. AVL Trees

A question to start

We need a BST that stays shallow no matter what order operations arrive in. Does one exist?

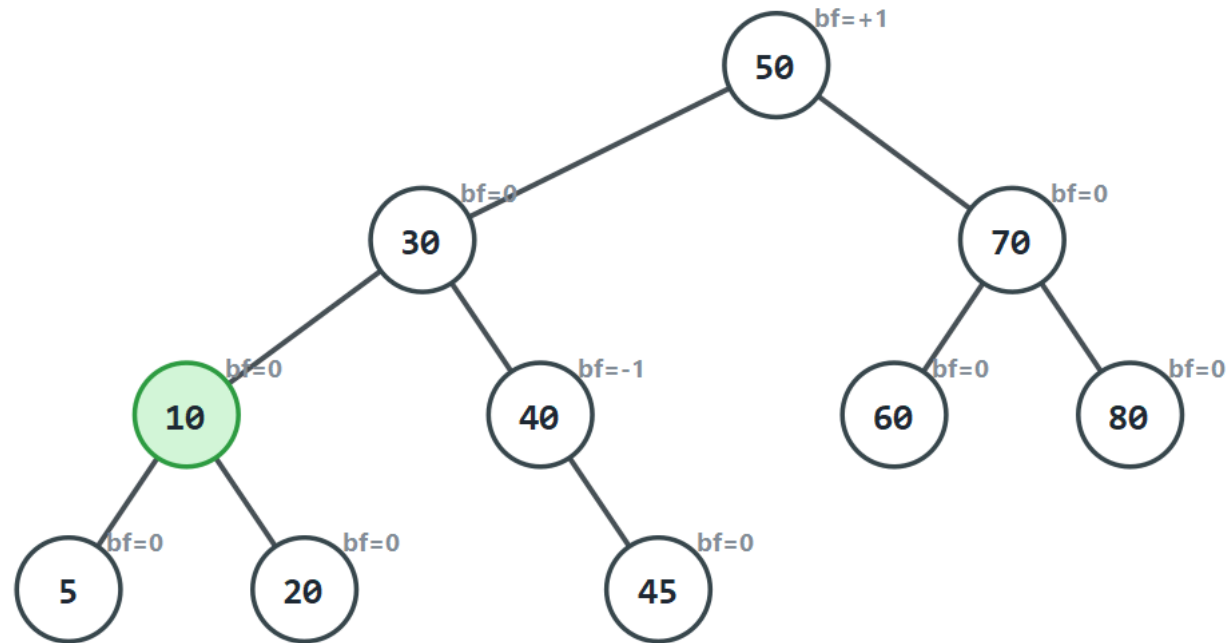
A short history

- **1962** — Georgy Adelson-Velsky & Evgenii Landis
- "AVL" = their initials
- First self-balancing binary search tree ever published
- Idea: track a **balance factor** at every node

The four rotation cases

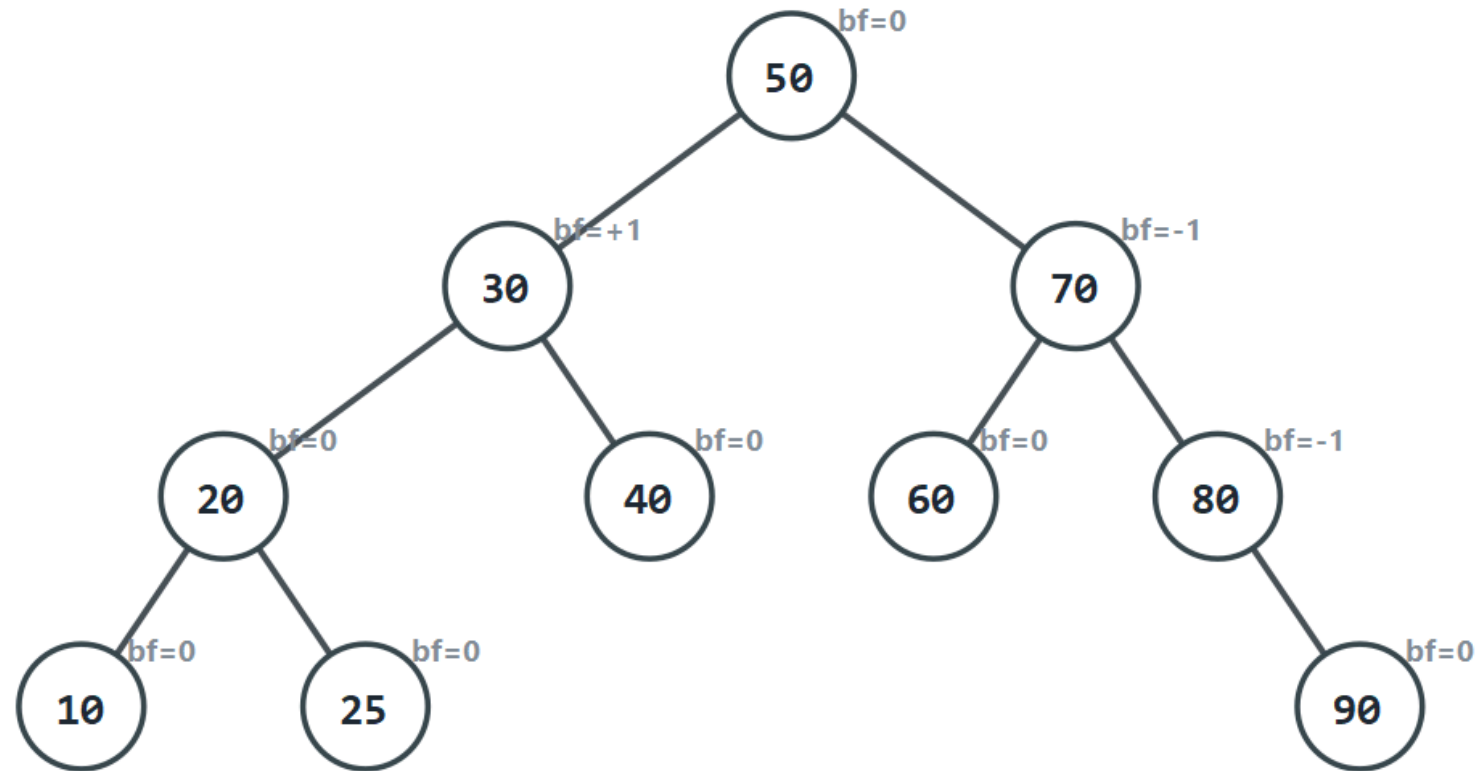
Case	Shape	Fix
LL	left-heavy, left child left-heavy	one right rotation
RR	right-heavy, right child right-heavy	one left rotation
LR	left-heavy, left child right-heavy	rotate left child left, then right
RL	right-heavy, right child left-heavy	rotate right child right, then left

AVL: the four rebalancing cases



Done. After one insertion, AVL needs at most ONE rotation (LL/RR) or ONE double rotation (LR/RL) — never more; that is why insert stays $O(\log n)$.

No rotation needed



Done. After one insertion, AVL needs at most ONE rotation (LL/RR) or ONE double rotation (LR/RL) — never more; that is why insert stays $O(\log n)$.

Mini question

After ONE insertion, how many rotations can an AVL tree need, at most?

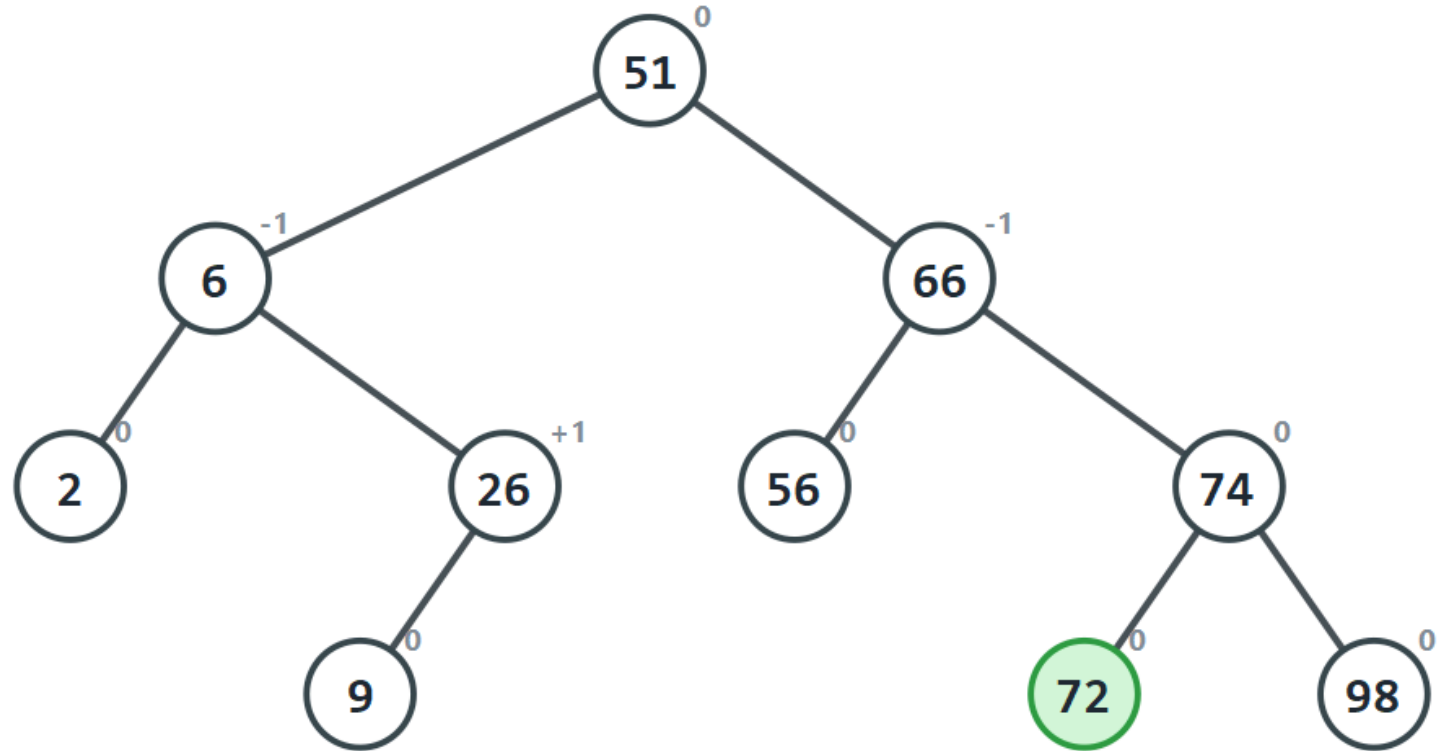
Mini answer

- At most ONE single rotation, or ONE double rotation
- The fix restores the subtree's PRE-insertion height exactly
- So no ancestor further up can have become unbalanced
- Insert never needs to propagate a fix upward

AVL insert — the idea

- Plain recursive BST insert
- On the way back UP the call stack: `update_height` , then `rebalance`
- Check happens at EVERY level as recursion unwinds
- First (and only) unbalanced node found is fixed immediately

AVL tree: insert



Done: 10 keys inserted, the tree stayed balanced at every step. insert is always $O(\log n)$: the comparisons plus at most one (double) rotation.

Code — rebalance()

```
Node *rebalance(Node *n) {  
    int bf = height(n->left) - height(n->right);  
    if (bf > 1 && height(n->left->left)  
        >= height(n->left->right))  
        return rotate_right(n);  
    if (bf > 1) {  
        n->left = rotate_left(n->left);  
        return rotate_right(n);  
    }  
}
```

Expected output — ascending keys

```
insert(10): height = 3
```

Section 1.4's plain BST reached **height 9** for the SAME 10 ascending keys.

Why AVL is $O(\log n)$

- Height never exceeds $1.44 \cdot \log_2(n + 2)$
- Every operation is $O(\log n)$ — WORST case, not just average
- insert: at most one (double) rotation
- delete: up to $O(\log n)$ rotations, but each is $O(1)$

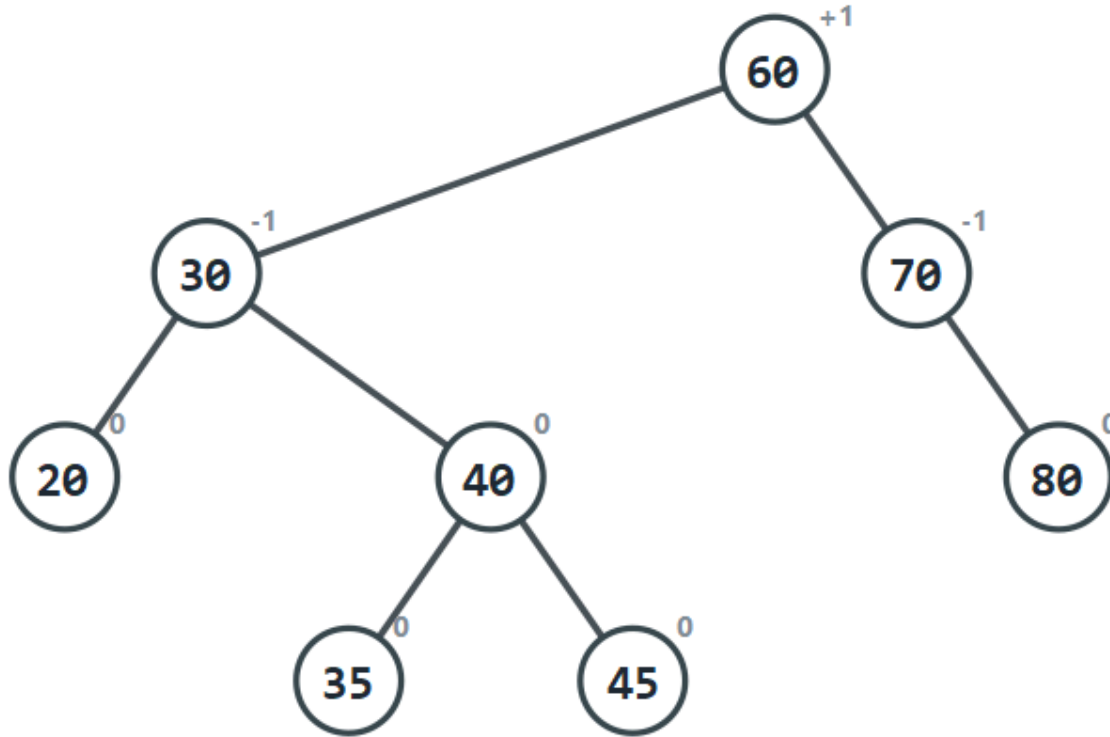
Common mistake

- Forgetting `update_height` before checking balance factor
- `rebalance` then reads a STALE height
- Deciding LL vs LR by the just-inserted key (breaks for delete)
- The balance-factor-based decision works for both insert AND delete

AVL delete — rebalancing can cascade

- Reuses section 1.4's exact splice logic (leaf/one/two children)
- Difference: `rebalance` runs at EVERY ancestor, not just the first
- A delete can shrink a subtree's height
- That shrink can keep propagating all the way to the root

AVL tree: delete



Done: 4 deletes processed. The tree stayed AVL throughout (bf always -1..+1). delete is still $O(\log n)$, but unlike a single insert, it can rotate at more than one level.

Expected output

```
delete(90): height = 3, inorder = 10 20 25 30 ...  
delete(45): height = 3, inorder = 10 20 25 30 ...
```

Height stays 3 across all six deletes — always rebalanced.

Why AVL delete is still $O(\log n)$

- Up to $O(\log n)$ rotations — one per ancestor level, worst case
- Each individual rotation is still $O(1)$
- $O(\log n)$ rotations $\times O(1)$ each = $O(\log n)$ total
- Same asymptotic bound as insert, just a bigger constant

Common mistake

- Assuming delete, like insert, needs only one rotation
- The "hard" scenario is deliberately built to disprove this
- Forgetting `free()` the spliced node in C (a leak)
- Not testing "delete every key down to empty" explicitly

Mini question

An AVL tree of height h has AT LEAST how many nodes?

Mini answer

- $N(h) = 1 + N(h-1) + N(h-2)$ — the Fibonacci recurrence
- $N(h)$ grows EXPONENTIALLY in h
- So h grows only LOGARITHMICALLY in n
- That is the whole proof, in one line

3. Red-Black Trees

A question to start

AVL's strict balance factor can require rebalancing on almost every insertion. Is there a looser rule?

A short history

- **1972** — Rudolf Bayer: "symmetric binary B-trees"
- **1978** — Guibas & Sedgewick: the "red-black" name
- Modern insertion algorithm dates from 1978
- Four simple, LOCAL rules instead of exact height comparison

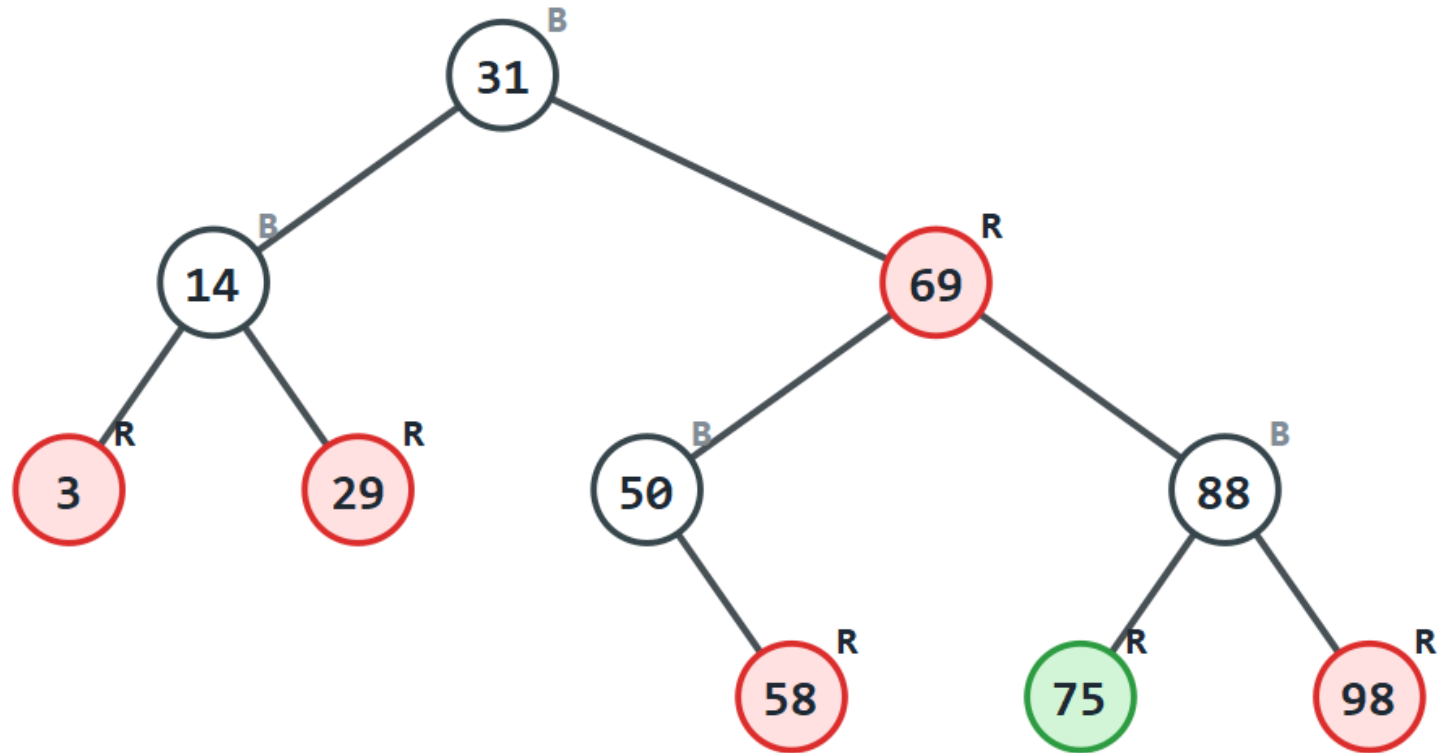
The four rules

- Every node is **red** or **black**
- The root is always **black**
- A red node never has a red child ("no two reds in a row")
- Every root-to-NULL path has the same **black-height**

The three fixup cases

Case	Situation	Fix
1	Uncle is RED	recolor parent+uncle black, grandparent red, continue up
2	Uncle black, "triangle"	rotate parent, reduces to case 3
3	Uncle black, "line"	rotate grandparent + recolor, done

Red-black tree: insert



Done: 10 keys inserted. Every fixup carries the red-red violation upward for at most $O(\log n)$ steps, then finishes with at most one (single or double) rotation — insert stays $O(\log n)$.

Code — fixup(), case 1

```
while (z->parent && z->parent->color == RED) {  
    Node *p = z->parent, *g = p->parent;  
    Node *u = (p == g->left) ? g->right  
                : g->left;  
    if (u && u->color == RED) {  
        p->color = BLACK; u->color = BLACK;  
        g->color = RED; z = g; continue;  
    }  
}
```

Expected output

```
insert(3): root = 3, bh = 1, inorder = 3B  
insert(69): root = 3, bh = 1, inorder = 3B 69R  
insert(31): root = 31, bh = 1, inorder = 3R 31B 69R
```

Why red-black is $O(\log n)$

- Height never exceeds $2 \cdot \log_2(n + 1)$
- Proof: no path can be more than twice the shortest
- (red nodes can never be adjacent)
- Slightly looser than AVL's bound, fewer rotations in practice

Common mistake

- Forgetting rule 2 after the fixup loop ends
- Case 1 can walk the violation all the way to the root
- The unconditional `root->color = BLACK;` at the end is NOT optional
- Confusing "uncle" with the new node's own sibling

Mini question

Why can inserting a RED leaf never break rule 4 (equal black-height), only rule 3?

Mini answer

- A red leaf contributes 0 to any path's black-node count
- Every root-to-leaf black-height stays exactly what it was
- Only rule 3 ("no two reds") can break, and only if the parent is red
- That is exactly the situation fixup is designed to repair

4. Splay Trees

A question to start

AVL and red-black pay bookkeeping cost on EVERY node,
for EVERY operation — even rarely-touched keys. A different way?

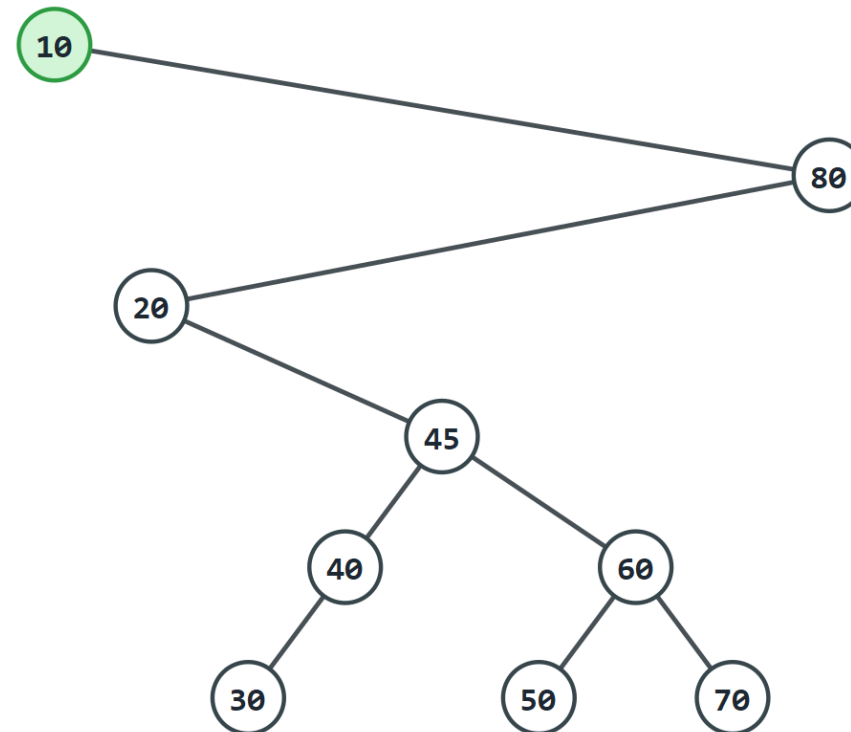
A short history

- **1985** — Daniel Sleator & Robert Tarjan
- "Self-adjusting binary search trees"
- No balance information stored at all — zero extra memory per node
- Instead: every access reshapes the tree

The idea — move to the root

Move	When	What happens
zig	parent is the root	one single rotation
zig-zig	node & parent both left (or both right) children	rotate parent, then node
zig-zag	node & parent on opposite sides	rotate node twice

Splay tree: access



Done: 12 accesses. A splay tree's guarantee is not worst-case, it is AMORTIZED: any single access can be $O(n)$, but m accesses together cost $O(m \log n) \rightarrow O(\log n)$ on average.

Code — splay()

```
void splay(Node *x) {  
    while (x->parent != NULL) {  
        Node *p = x->parent, *g = p->parent;  
        if (g == NULL) { rotate_up(x); }  
        else if ((x == p->left) == (p == g->left))  
            { rotate_up(p); rotate_up(x); }  
        else { rotate_up(x); rotate_up(x); }  
    }  
}
```

Expected output

```
access(50): root = 50, inorder = 50  
access(20): root = 20, inorder = 10 20 30 40 45 50 60 70 80
```

The just-accessed key is ALWAYS the new root.

Why splay is $O(\log n)$ amortized

- No single access is guaranteed $O(\log n)$ — can be $O(n)$
- ANY sequence of m accesses costs $O(m \log n)$ total
- $O(\log n)$ amortized per access
- Adapts automatically to a "hot" working set

Common mistake

- Implementing zig-zig as two SEPARATE single rotations ("naive splay")
- Valid tree operation, but loses the amortized guarantee
- Forgetting access on a MISSING key still splays something
- Here: the newly inserted node itself

Mini question

After many accesses to ONE key, how deep is every OTHER key, roughly?

Mini answer

- That one key sits at the root every time
- Every OTHER key's depth is barely affected
- Splay only reshuffles nodes ALONG the accessed path
- No "global" balance guarantee — only a local, per-access one

5. 2-3 Trees

A question to start

Every tree so far fixes imbalance AFTER it happens.
What if imbalance were structurally IMPOSSIBLE?

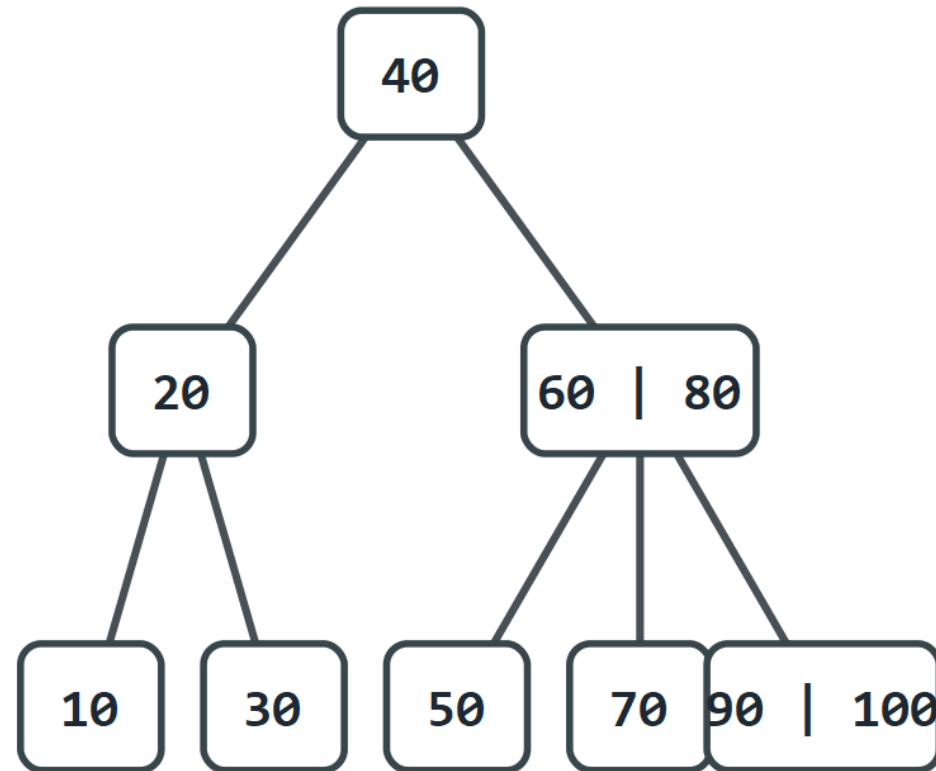
A short history

- **1972** — Rudolf Bayer & Edward McCreight
- A stepping stone to the B-tree (Week 14, disk-backed files)
- Node holds 1 key (2-node) or 2 keys (3-node)
- Every leaf sits at EXACTLY the same depth, always

The idea — overflow and split

- New key inserted into the correct leaf, sorted
- Leaf already had 2 keys → now has 3, temporarily: **overflow**
- Split into two 2-nodes; MIDDLE key promoted to the parent
- Parent can overflow the same way — cascades upward

2-3 tree: insert



Done: 10 keys inserted. Every leaf is still at the SAME depth — a 2-3 tree never becomes unbalanced, because growth always happens at the root.

Code — the overflow loop

```
while (node->nkeys == 3) {
    split_node(node, &left, &right, &promoted);
    if (depth == 0) {
        /* root split: height + 1 */
        return new_root(promoted, left, right);
    }
    Node *parent = path[--depth];
    replace_with_split(parent, node,
                       promoted, left, right);
    node = parent;
}
```

Expected output

```
insert(20): height = 0, level-order = [10,20]  
insert(30): height = 1, level-order = [20] [10] [30]
```

Why 2-3 tree insert is $O(\log n)$

- Every leaf at the same depth $h \rightarrow h = O(\log n)$
- Walk down: $O(h)$; splits cascade up: at most $O(h)$, each $O(1)$
- Zero rotations, ever — unlike every other tree today
- Direct ancestor of Week 14's B-tree

Common mistake

- Sizing `child[]` for only 3 slots (the steady-state max)
- Mid-overflow, a node briefly needs 4 children — real buffer overflow
- Splitting a node without freeing the old shell afterward
- Promoting the wrong key (must be the MIDDLE one)

Mini question

Why does a 2-3 tree's height grow ONLY at the root, never partway down?

Mini answer

- A split only responds to ITS OWN node's overflow
- The promoted key goes to ITS OWN parent, never a sibling
- The cascade only ever travels straight up one path
- The only place it can run out of "parent" is the root

6. Segment Trees

A question to start

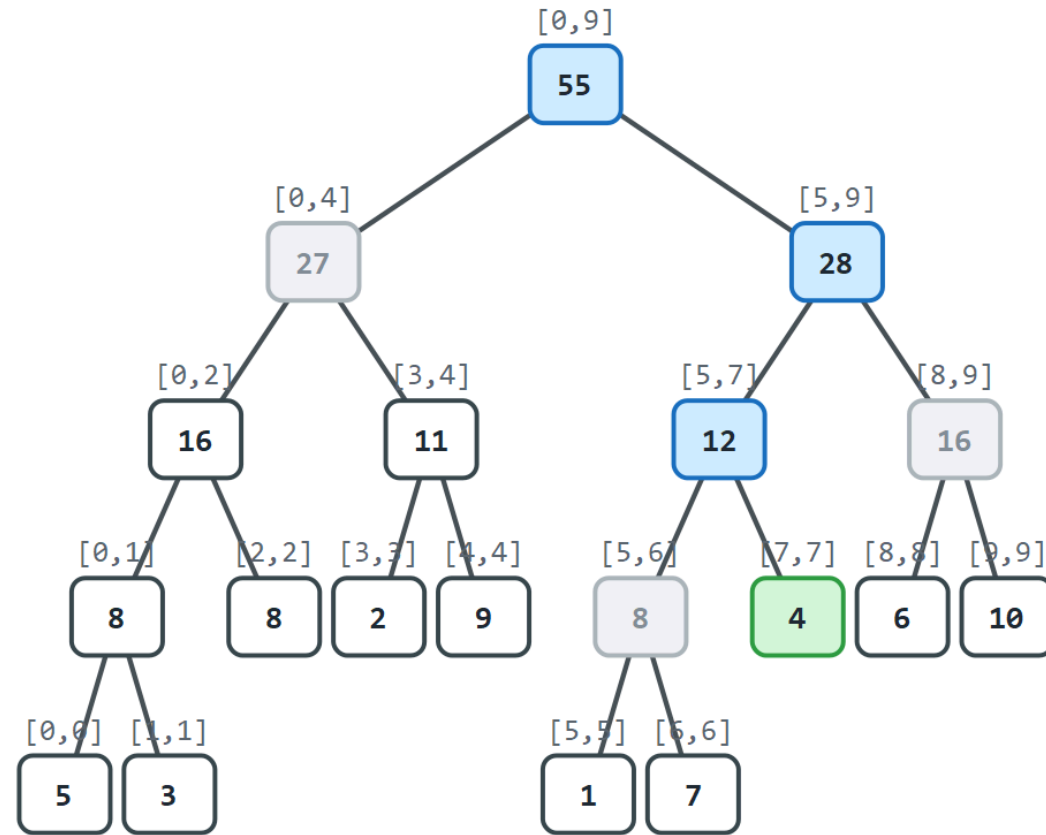
"What is the SUM of every value between index l and r ?"

A loop answers in $O(n)$. Many such queries — can we do better?

The idea — precompute every range once

- Built once over a fixed-size array
- Node i responsible for range $[lo, hi]$; children $2i, 2i+1$
- Leaf holds one value; internal node holds sum of its two children
- Query walks down: outside $\rightarrow 0$, inside $\rightarrow$ precomputed sum, partial $\rightarrow$ recurse both

Segment tree: build and query



Done: 3 queries. Every query visits at most $O(\log n)$ nodes — a big win over a plain loop's $O(n)$, especially with many queries.

Code — query()

```
long query(int i, int lo, int hi, int l, int r) {  
    if (r < lo || hi < l)    return 0;  
    if (l <= lo && hi <= r) return tree[i];  
    int mid = (lo + hi) / 2;  
    return query(2*i,    lo,    mid, l, r)  
        + query(2*i+1, mid + 1, hi,  l, r);  
}
```

Expected output

```
query(0,9) = 55  
query(2,5) = 20  
query(7,7) = 4
```

Why segment tree query is $O(\log n)$

- build(): $O(n)$ total — visits every node once
- Each level: at most TWO "partially overlapping" nodes
- Every other node at that level: answered or pruned immediately
- Total work proportional to height: $O(\log n)$

A structural difference from sections 1–5

- Segment tree SHAPE depends only on n , never on data values
- Every BST-family tree's shape depends on VALUE comparisons
- A segment tree can never degenerate the way section 1.4's BST did
- No insertion-order problem exists here at all

Common mistake

- Sizing the array as $2n$ instead of $4n$
- Confusing "fully inside" with "fully outside" (swapped conditions)
- Rebuilding the WHOLE tree ($O(n)$) for one single-point update
- A dedicated $O(\log n)$ point-update function is the natural fix

Mini question

Why is query's cost $O(\log n)$, not $O(\log n)$ TIMES the number of nodes at each level?

Mini answer

- At most TWO nodes per level are "partially overlapping"
- Every other node: fully inside (done) or fully outside (pruned)
- Total work proportional to height, not width
- $O(\log n)$, not $O(\log n) \cdot O(n)$

7. Fenwick Trees

A question to start

A segment tree needs up to $4n$ explicit nodes.
Could a PLAIN ARRAY answer prefix sums just as fast?

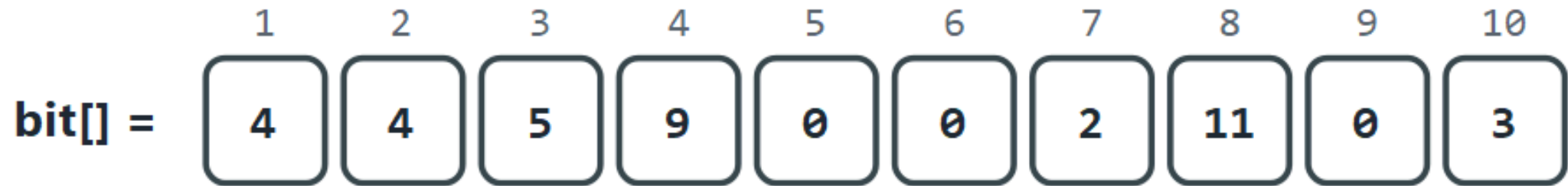
A short history

- 1994 — Peter Fenwick
- "A new data structure for cumulative frequency tables"
- No tree pointers, no recursion required
- One array, one arithmetic trick: `i & -i`

The idea — $i \& -i$ isolates the lowest set bit

- `bit[i]` holds the sum of a range of size `$i \& -i$` ending at `i`
- `update(i, delta)`: walk UP, `$i += i \& -i$`
- `query(i)`: prefix sum 1..i, walk DOWN, `$i -= i \& -i$`
- Both walks: $O(\log n)$ steps, no tree structure needed

Fenwick tree: update and query



Done: 8 operations. Every update and every query takes at most $O(\log n)$ steps — $i \& -i$ is a simple arithmetic trick that isolates the rightmost 1 bit in binary.

The i & $-i$ chain length, in isolation



Done: 2 operations. Every update and every query takes at most $O(\log n)$ steps — i & $-i$ is a simple arithmetic trick that isolates the rightmost 1 bit in binary.

Code — update() and query()

```
void update(int i, int delta) {  
    while (i <= n) { bit[i] += delta; i += i & (-i); }  
}  
int query(int i) {  
    int sum = 0;  
    while (i > 0) { sum += bit[i]; i -= i & (-i); }  
    return sum;  
}
```

Expected output

```
update(3, 5)  
update(7, 2)  
query(10) = 7
```

Why Fenwick is $O(\log n)$, $O(n)$ space

- i & $-i$ at least doubles (update) or halves (query) distance to the boundary
- At most $\text{floor}(\log_2(n)) + 1$ iterations, either direction
- Space: one plain int array — dramatically less than a segment tree
- Preferred in practice when the array's size will not change

Common mistake

- Using 0-indexing — `i & -i` needs index 0 to be all-zero bits
- Fenwick trees are ALWAYS 1-indexed
- Writing a home-made "negation" instead of the language's `-`
- Reaching for Fenwick on range-MIN/MAX queries (does not work)

Mini question

For $n=16$, why does `update(1)` take exactly 5 steps
($i = 1, 2, 4, 8, 16$)?

Mini answer

- `i += i & -i` : $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16$, then the loop stops (i would exceed n)
- Five cells touched, each responsible for a range including index 1
- query(15) instead takes 4 steps: one per 1-bit in 15's binary (1111)
- Update and query walk opposite directions, different bit patterns

8. Choosing a Technique

Comparison — balanced BSTs

Structure	Worst-case height	Rebalance cost
Plain BST	$O(n)$	none
AVL	$O(\log n)$, tightest	≤ 1 rotation/insert
Red-black	$O(\log n)$, looser	fewer rotations avg.
Splay	$O(\log n)$ amortized	full splay/access

Comparison — structural & range trees

Structure	Rebalance cost	Best for
2-3 tree	node splits, no rotations	bridge to Week 14's B-tree
Segment tree	none after build	many range sum/min/max queries
Fenwick tree	none	range sum + frequent point updates

Choosing — by workload

- **Lookup-heavy** → AVL (tightest bound)
- **Mixed insert/delete/search** → red-black (fewer rotations)
- **Skewed "hot key" access** → splay (adapts, low memory)
- **Range questions** → segment tree or Fenwick tree

Mini question

A colleague says "always use red-black, it's the most balanced and library-tested". What would you ask first?

Mini answer

- Is the workload lookup-heavy, or a mixed insert/delete/search?
- Is access skewed toward a small hot-key set?
- Are the questions about RANGES, not single keys?
- Will this feed into a disk-backed structure later (Week 14)?

Summary

- BST: `insert` / `search` / `delete` all $O(h)$ — but h depends on ORDER
- Sorted input degenerates a BST to a chain: $O(n)$, no better than a list
- AVL, red-black, splay, 2-3 tree: four different fixes, four trade-offs
- Segment tree, Fenwick tree: $O(\log n)$ answers to RANGE questions

Exercises preview

- Prove a BST of height h has at most $2^{(h+1)} - 1$ nodes
- Trace red-black's fixup cases by hand on the "hard" scenario
- Sketch what changes to turn a segment tree into range-MIN
- Full list of 10 exercises: this week's notes

Self-check quiz preview

- Why does AVL insert need at most one rotation, but delete can cascade?
- Why can a Fenwick tree not support range-minimum queries?
- Why is a segment tree's shape independent of the data?
- Full 10-question quiz with answers: this week's notes

Looking ahead

- **Week 12** — strings: matching algorithms, and the **trie**
- A trie stores strings by shared PREFIX, not numeric comparison
- **Week 13** — direct and sequential file organization
- **Week 14** — the **B-tree**: this week's 2-3 tree, generalized for disk

Questions?

CEN207 Data Structures — Week 11

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027